Journal of Physics:
Conference Series

PURPOSE-LED PUBLISHING™

PAPER • **OPEN ACCESS**

Research on robotic arm path planning based on an improved bidirectional RRT* algorithm

To cite this article: Guochen Niu *et al* 2025 *J. Phys.: Conf. Ser.* **3120** 012031

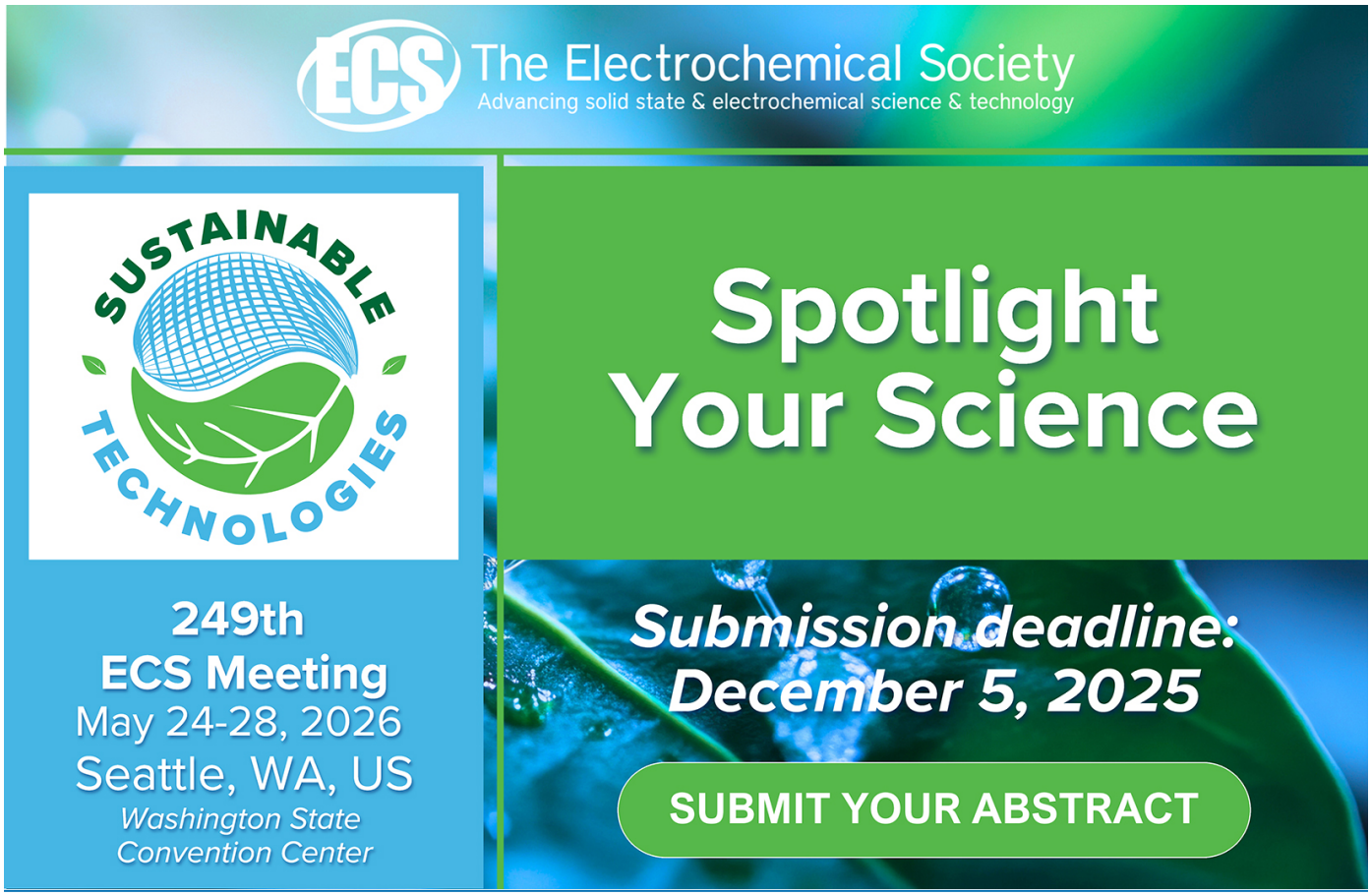
View the article online for updates and enhancements.

You may also like

- RSP-UV: real-time sampling-based path planning method for unmanned vehicles
Rui Zhou, Chuanwei Zhang, Ruiqi Zhao et al.

- Research on robotic arm path planning based on improved bidirectional BI-APF-RRT*
Xuefu Dong, Hailong Ge, Jiabin Sun et al.

- Research on robotic arm path planning method based on two-stage RRT* optimization algorithm
Entie Qi, Tonglin Zhang, Liying Zhao et al.



This content was downloaded from IP address 23.165.184.112 on 01/12/2025 at 09:41

Research on robotic arm path planning based on an improved bidirectional RRT* algorithm

Guochen Niu*, Biao Tang and Xue Wang

Institute of Robotics, Civil Aviation University of China, Tianjin 300300, China

niu_guochen@139.com; 15633908703@163.com; 17343335660@163.com

Abstract. To enable efficient, smooth, and obstacle-free path planning for robotic arms during cleaning operations, this study introduces an enhanced bidirectional RRT* algorithm grounded in arm kinematics. Addressing limitations like poor goal orientation and winding trajectories, the method integrates strategies such as collision checking, target biasing, adaptive step sizing, and path pruning. A cubic B-spline is then used to smooth the resulting path. Experimental validation confirms the method's effectiveness in improving planning efficiency, convergence rate, and path quality.

1. Introduction

Robotic arm path planning serves as a fundamental technology for autonomous robotic tasks. Its primary objective is to generate efficient, collision-free, and dynamically executable trajectories in environments containing obstacles. Consequently, devising planning algorithms adaptable to diverse surroundings holds both academic and practical significance. Presently, mainstream approaches fall into three broad categories.

(1) Grid-based search methods like A* and Dijkstra operate on discretized environments using breadth-first traversal. While they ensure comprehensive exploration, they encounter exponential growth in grid size as complexity rises, resulting in high memory consumption and reduced efficiency.

(2) Swarm intelligence methods—such as Particle Swarm Optimization, Genetic Algorithms, and Ant Colony Optimization—emulate collective biological behavior, using parallel mechanisms to approach global solutions. However, they are memory-intensive, require large-scale populations, and are highly dependent on fine-tuned parameters, making them prone to slow convergence when misconfigured.

(3) Sampling-based planners—primarily Probabilistic Roadmaps [1] and RRT [2]—are widely used for high-dimensional problems. RRT generally outperforms PRM in single-query scenarios but exhibits issues like delayed convergence and suboptimal path quality [3]. RRT builds a search tree through random sampling without prior environment processing, making it suitable for high-dimensional systems like robotic arms. Its fast search speed and constraint handling are advantages, but random sampling leads to many invalid nodes, increasing computation time and producing less smooth paths.

To address these issues, various improvements have been proposed. Karaman et al. [4] introduced parent rewiring for asymptotic optimality, but the added complexity lowers efficiency in 3D [5]. Liao et al. [6] proposed F-RRT using the triangle inequality for faster parent selection, yet a fallback to other variants increases time in some cases. Sepehre et al. [7] combined RRT with Artificial Potential Fields for better directionality, but risk of local optima. Huang et al. [8] developed Agile-RRT* with adaptive goal biasing to speed convergence, limited to 2D. Dai et al. [9] proposed a bidirectional RRT guided by potential fields (NPB-RRT-DC), improving safety margins and efficiency by 45%, but lacking dynamic adaptability.

Content from this work may be used under the terms of the Creative Commons Attribution 4.0 licence. Any further distribution of this work must maintain attribution to the author(s) and the title of the work, journal citation and DOI.
Published under licence by IOP Publishing Ltd

In summary, while RRT and variants perform well, they still face blind sampling, tortuous paths, and slow convergence. Since the robotic arm operates in a small workspace, further RRT improvements are needed to boost efficiency and adaptability for lighting fixture cleaning tasks. This study adopts a goal-directed strategy, an adaptive step-size adjustment strategy, and a pruning optimization method to improve the algorithm's performance. Specifically, the goal-biased strategy employs the Halton sequence for sampling, which maintains randomness while ensuring more uniform 3D point distribution, enhancing search efficiency. The adaptive step-size strategy dynamically adjusts the extension step based on the real-time distance between the tree node and the target, balancing a fast goal approach and local obstacle avoidance. The pruning optimization strategy simplifies the initial path by replacing segmented paths with straight lines when collision-free, effectively removing redundant points and shortening path length. In addition, a path smoothing process is applied to enhance the smoothness and feasibility of the planned paths.

2. Constrained Path Planning for Robotic Arms in Complex Environments

2.1. Collision Detection for Robotic Arms

Besides ensuring the end-effector reaches the desired pose, robotic arm path planning must avoid collisions with obstacles to ensure safety. Typically, collision detection checks rigid body interactions. To simplify this, an envelope-based method approximates arm links as cylinders and obstacles as spheres, reducing collision checks to cylinder-sphere spatial relationships. The detailed process is as follows: for each link of the robotic arm, its shape is approximated by a cylinder with a certain radius. Similarly, obstacles in the environment are represented as spheres with specific radii. Collision is determined by calculating the minimal distance between the sphere center and the cylindrical surface—if this distance is less than the combined radii of both, a collision is identified; otherwise, the system remains collision-free. This method greatly simplifies computation and improves efficiency.

While maintaining accuracy, this method enables fast collision detection, making it suitable for complex path planning. The bounding sphere is defined as follows:

$$B = \left\{ (x, y, z) \mid (x-a)^2 + (y-b)^2 + (z-c)^2 \le R^2 \right\} \tag{1}$$

In the above equation, a point set B in 3D space is given, where each point is represented by coordinates (x, y, z). Let (a, b, c) be the center coordinates of the minimum bounding sphere enclosing the point set, and let R be the radius of this bounding sphere. In the collision detection model shown in Figure 1, links i and $i+1$ are simplified as a single cylinder with a radius of i, and its two endpoints are denoted as E_i and F_i. Obstacles are simplified as spheres, with center coordinates O and radius R. If the shortest distance d_i from the obstacle center O to link i satisfies the condition $d_i < \omega_i + R$, a collision is determined to have occurred; otherwise, no collision is detected.

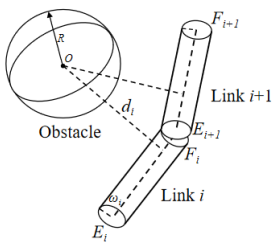


Figure 1. Collision Detection Model for Robotic Arm.

As the robotic arm operates in a 3D environment, this work employs the Bidirectional RRT* algorithm to generate feasible trajectories. The Bidirectional RRT* algorithm constructs two RRT trees, T_1 and T_2, starting respectively from the initial point Q_{init} and the goal point Q_{goal}, to perform path planning. First, Q_{init} and Q_{goal} are set as the root nodes of T_1 and T_2, respectively. Then, a random point Q_{rand} is sampled within the workspace. In T_1, the nearest node to Q_{rand}, denoted as Q_{near1}, is found, and

a new node Q_{new1} is generated by extending a fixed step size toward Q_{rand}. Collision detection is performed for Q_{new1}, and if no collision occurs, it is added to T_1. The same process is repeated for T_2, generating a new node Q_{new2}, which is then added to T_2. Finally, the algorithm checks whether the newly generated nodes from T_1 and T_2 can be connected without collision. If a collision-free connection is found, the path planning process is completed. These steps are repeated until the two trees meet or the maximum number of iterations is reached. The final collision-free path from Q_{init} to Q_{goal} is generated by connecting the meeting points of the two trees. Figure 2 illustrates the expansion process of the Bidirectional RRT* algorithm in generating a collision-free path.

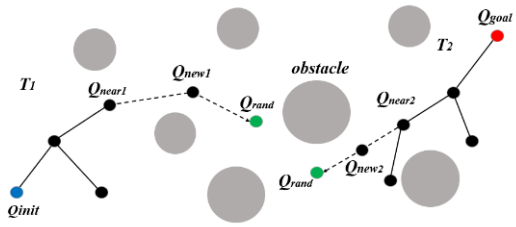


Figure 2. Expansion Process of the Bidirectional RRT* Algorithm.

2.2. Improved Bidirectional RRT Algorithm*

To meet the operational requirements of embedded light fixture cleaning, it is necessary to improve the Bidirectional RRT* algorithm in multiple aspects, including the sampling strategy, random node generation strategy, and node adjustment strategy. These enhancements aim to improve the cleaning efficiency during the embedded light fixture cleaning process.

(1). Goal-Biased Strategy

The traditional Bidirectional RRT* and its variants use pseudo-random sequences for sampling, causing uneven spatial distribution and dense local sampling, which leads to excessive iterations and low search efficiency. To improve this, this section employs the Halton sequence—a low-discrepancy sequence based on the Van der Corput sequence [10]—for sampling. The Halton sequence maintains randomness while ensuring more uniform 3D point distribution, enhancing search efficiency. The Van der Corput sequence is defined as follows.

$$i = \sum_{l=0}^{M=1} a_l(i) b^l \tag{2}$$

Let b be a positive integer. For any integer i, represent it in base-b, and arrange its digits $a_l(i)$ to form a vector. Then, the Van der Corput sequence can be expressed as:

$$\Phi_b(i) = (b^{-1} \cdots b^{-M})(a_0(i) \cdots a_{M-1}(i))^T = \sum_{i=0}^{M=1} a_i(i) b^{-l-1} \tag{3}$$

In conjunction with Equation (2), the definition of the Halton sequence is given as follows:

$$X_i := (\Phi_{b_1}(i), \Phi_{b_2}(i), \Phi_{b_3}(i)) \tag{4}$$

To further improve the convergence speed of the Bidirectional RRT* algorithm and reduce path cost, this section introduces a goal-biased property to optimize the tree nodes. Taking tree T_1 as an example, the implementation process of the goal-biased Bidirectional RRT* algorithm is described. In the three-dimensional Halton sequence, the norm of the point $\|Halton(i)\|$ is used as a sampling guidance parameter. Since a larger norm $\|Halton(i)\|$ indicates that the point is closer to the target region (e.g., the coordinate origin), points with larger norms are preferentially selected as the initial random sampling points Q_{rand1}.

$$Q_{rand1} = \begin{cases} Halton(Q_i), & \| Halton.Z \| < 0.5 \\ Q_{goal1}, & \| Halton.Z \| \geq 0.5 \end{cases} \tag{5}$$

According to Equation (5), the initial random point Q_{rand1} of tree T_1 is generated within the reachable space to enhance its goal bias. The initial random point Q_{rand2} of tree T_2 is generated using the same strategy.

(2) Adaptive Step-Size Strategy

To balance a fast goal approach and local obstacle avoidance, a distance-feedback adaptive step-size strategy is introduced. It dynamically adjusts the extension step based on the real-time distance d between the tree node and the target. Given preset thresholds [d_{min}, d_{max}] and step limits s_{min}, s_{max}, a nonlinear interpolation adjusts the step size smoothly within this range. This avoids overly large steps, causing collisions, and overly small steps, causing long planning times. The step-size calculation formula is:

$$s = \begin{cases} s_{min} & d \le d_{min} \\ s_{min} + \left(\dfrac{d - d_{min}}{d_{max} - d_{min}} \right)^k (s_{max} - s_{min}) & d_{min} \le d \le d_{max} \\ s_{max} & d > d_{\max} \end{cases} \tag{6}$$

Here, s is the current step size, d is the real-time distance, and k is the adjustment factor. When $d \le d_{min}$, the node is close to the target or in an obstacle-dense area, and the step size is set to s_{min} to enhance obstacle avoidance precision and safety. When $d \ge d_{max}$, the environment is relatively open, and the maximum step size s_{max} is used to speed up long-range search. For $d_{min} < d < d_{max}$, a power function interpolation is applied to smoothly transition the step size and avoid abrupt changes that could destabilize planning. The parameter k controls the rate of step-size variation: when $k > 1$, the step-size increases gradually, which is suitable for dense environments; when $k < 1$, the step-size increases more rapidly, favoring fast exploration in sparse regions.

(3) Pruning Optimization Strategy

In the bidirectional RRT* algorithm, the initial path usually consists of multiple piecewise linear segments. As shown in Figures 3, to simplify it, a pruning method replaces segmented paths with straight lines when collision-free. Starting from the initial point, the algorithm checks for obstacles to the goal; if none are found, the goal's parent is set to the start. Otherwise, the goal is replaced iteratively by its parent until a valid connection is found. During this process, a safety margin is included during collision checks to ensure clearance and allow for later smoothing. The specific steps are as follows: First, the algorithm identifies the parent nodes of each node along the path. Then, it attempts to connect each node directly to its grandparent node, checking for collisions along the potential new path segment. If no collision is detected, the parent node is removed, and the path is updated. This process continues until no more parent nodes can be removed. By doing so, the path is simplified while remaining collision-free.

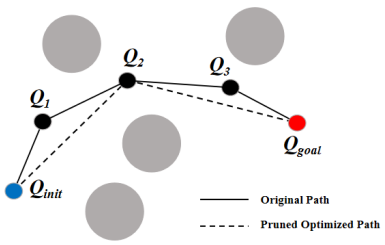


Figure 3. Pruning Optimization Diagram.

(4) Path Smoothing Processing

In the path smoothing phase, to address the issue of sharp turns in the pruned and optimized path, this study employs cubic B-spline curves for smoothing optimization. First, key points of the path are selected as initial control points, and the curve's exact matching at the start and end points is ensured through node vector parameterization. Next, the De Boor recursive algorithm is used to compute the cubic B-spline basis functions, guaranteeing C^2 continuity of the curve. Finally, control point

optimization via inverse calculation and discrete sampling verification is performed to achieve motion smoothness while preserving the path's geometric features. This method allows path shape optimization through localized control point adjustments, maintaining the local controllability of B-spline curves while meeting the acceleration continuity requirements for robotic arm motion, thereby effectively enhancing path quality. The mathematical expression is as follows:

$$G(u) = \sum_{i=0}^{n} N_{i,p}(u) P_i \tag{7}$$

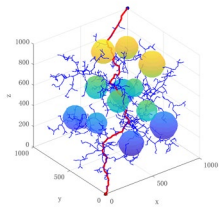
In the above equation, u is the parameter variable, ranging from 0 to 1. P_i represents the i_{th} control point, with a total of $n+1$ control points. $N_{i,p}(u)$ is the p_{th} degree B-spline basis function corresponding to control point P_i, and its specific expression is given as follows:

$$N_{i,p}(u) = \frac{1}{p!} \sum_{j=0}^{p-i} (-1)^j \times C_{n+1}^{j} \times (t - i - j + n)^p \tag{8}$$

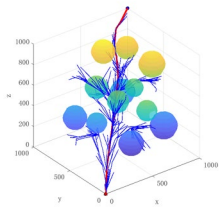
Smoothing is performed using a cubic B-spline curve, i.e., p = 3.

2.3. Simulation Experiment of the Improved Bi-directional RRT Algorithm*

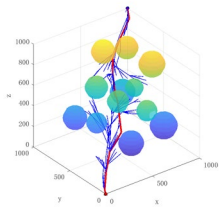
To demonstrate the advantages of the improved bidirectional RRT* algorithm, a comparative simulation was conducted against RRT, RRT*, and standard bidirectional RRT* in the same 3D environment (1000 × 1000 × 1000) with 11 obstacles. Each algorithm was run for up to 5000 iterations and repeated 100 times. Figures 4 and 5 show the simulation results and the relationship between iteration counts and path lengths.



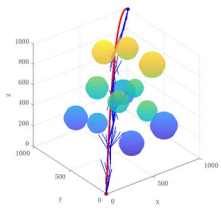
(a) RRT Search Process and Results



(b) RRT* Search Process and Results

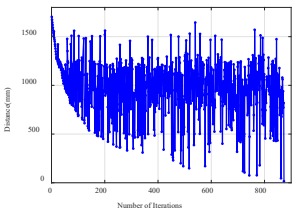


(c) Bi-RRT Search Process and Results

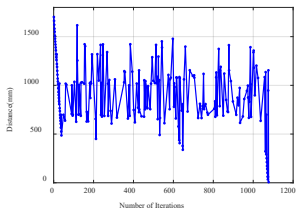


(d) Improved Bi-RRT Search Process and Results

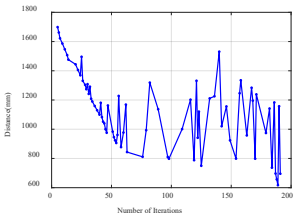
Figure 4. Path Planning Process and Results of Various Algorithms (Blue Lines Represent the Search Process, Red Lines Represent the Final Path).



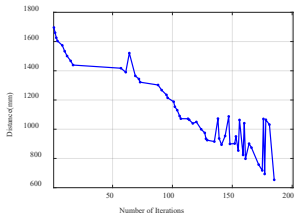
(a) Relationship Between Iteration Count and Distance in RRT



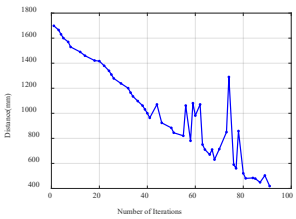
(b) Relationship Between Iteration Count and Distance in RRT*



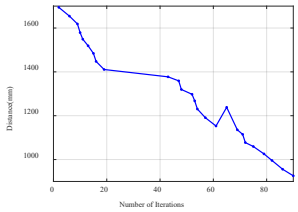
(c) Relationship Between Initial Tree Iteration Count and Distance in the Bi-RRT* Algorithm



(d) Relationship Between Goal Tree Iteration Count and Distance in the Bi-RRT* Algorithm



(e) Relationship Between Initial Tree Iteration Count and Distance in the Improved Bi-RRT* Algorithm



(f) Relationship Between Goal Tree Iteration Count and Distance in the Improved Bi-RRT* Algorithm

Figure 5. Relationship Between Algorithm Iteration Count and Path Length.

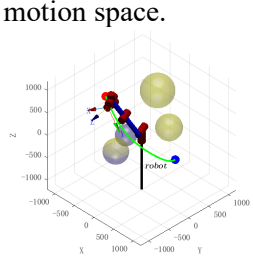
As shown in Figures 4-5, the improved Bi-RRT* outperforms conventional methods by integrating goal bias, adaptive step-size, and pruning optimization. It generates straighter paths with faster convergence (within 90 iterations), significantly reducing computational costs compared to unidirectional approaches. This makes it ideal for embedded lamp-cleaning path planning.

Table 1. Performance Comparison Table of Various Algorithms**.**

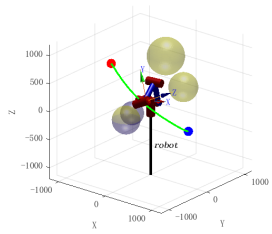
Algorithm	Average Planning Time (s)	Average Number of Iterations	Average Number of Valid Nodes	Average Path Length (mm)
RRT	50.14	1385.80	39.46	2337.61
RRT*	35.63	1003.62	13.46	1767.05
Bi-RRT	11.68	204.88	15.50	1998.27
Improved Bi-RRT*	5.68	192.64	3.14	1745.87

As shown in Table 1, the improved Bi-RRT* outperforms other algorithms significantly. It reduces planning time to 5.68 s (51.4% faster than Bi-RRT*), requires only 192.64 iterations (86% fewer than RRT), and generates shorter paths (1745.87 mm, 12.6% improvement over Bi-RRT*). These enhancements make it highly efficient for real-time robotic arm applications, such as aeronautical light cleaning.

To verify the effectiveness of the robotic arm's obstacle avoidance strategy, a 1000 × 1000 × 1000 Cartesian coordinate system experimental scenario was constructed in MATLAB. As shown in Figure 6, the origin of the robotic arm's coordinate system was set as the scene origin, with appropriate obstacles constraining its motion space.



(a) Initial position



(b) Intermediate position 1

(c) Intermediate position 2 (d) Target position

Figure 6. Obstacle avoidance process of the robotic arm (red dot as the start point, blue dot as the goal point).

Figure 6 demonstrates that integrating robotic arm pose constraints in Bi-RRT* ensures collision-free motion at every node. The results validate the algorithm's effectiveness in complex environments, proving its robust constraint-handling capability for robotic path planning.

3. Conclusion

To achieve collision-free path planning for the robotic arm during cleaning operations, the bidirectional RRT* algorithm was introduced for initial path searching. To address its limitations—such as weak goal bias and convoluted paths—this paper proposes several improvements, including a goal-directed strategy, an adaptive step-size mechanism, and pruning optimization. The resulting path is then smoothed to meet kinematic constraints. Comparative analysis was conducted in a 3D MATLAB environment against existing methods. Experimental results demonstrate that the improved bidirectional RRT* algorithm performs excellently in terms of planning speed, convergence, and path quality, effectively meeting the path planning requirements for robotic arms operating on embedded aeronautical lighting systems.

References

[1] Kavraki L E, Svestka P, Latombe J C, et al. (1996) Probabilistic roadmaps for path planning in high-dimensional configuration spaces. IEEE Transactions on Robotics and Automation, 12(4): 566–580.
[2] Lavalle S M. (1998) Rapidly-exploring random trees: A new tool for path planning. Computer Science Dept., 98(11).
[3] Tsianos K I, Sucan I A, Kavraki L E. (2007) Sampling-based robot motion planning: Towards realistic applications. Computer Science Review, 1(1): 2–11.
[4] Karaman S, Frazzoli E. (2011) Sampling-based algorithms for optimal motion planning. The International Journal of Robotics Research, 30(7): 846–894.
[5] Zhang W, Shan L, Chang L, et al. (2023) SVF-RRT: A stream-based VF-RRT for USVs path planning considering ocean currents. IEEE Robotics and Automation Letters, 8(4): 2413–2420.
[6] Liao B, Wan F, Hua Y, et al. (2021) F-RRT*: An improved path planning algorithm with improved initial solution and convergence rate. Expert Systems with Applications, 184: 115457.
[7] Sepehri A, Moghaddam A M. (2021) A motion planning algorithm for redundant manipulators using rapidly exploring randomized trees and artificial potential fields. IEEE Access, 9: 26059–26070.
[8] Huang C, Tang B, Guo Z, et al. (2024) Agile-RRT*: A Faster and More Robust Path Planner with Enhanced Initial Solution and Convergence Rate in Complex Environments. IEEE Access, 12: 58703–58714.
[9] Dai J, Zhang Y, Deng H. (2023) Novel potential guided bidirectional RRT* with direct connection strategy for path planning of redundant robot manipulators in joint space. IEEE Transactions on Industrial Electronics, 71(3): 2737–2747.
[10] Hofer R. (2018) Halton-type sequences in rational bases in the ring of rational integers and the ring of polynomials over a finite field. Mathematics and Computers in Simulation, 143: 78–88.